

C++ COMPETITIVE PROGRAMMING + DATA STRUCTURES + ALGORITHMS MASTER TEXTBOOK

*From Absolute Beginner to Advanced Competitive
Programmer*

PART VIII LINKED LISTS

*Singly, doubly & circular lists · Reversal · Fast & slow
pointers · Cycle detection · Merging*

This Part follows Part VII (Hashing). It is the first data structure in this book built directly from raw pointers rather than an STL container — everything from Part I Chapter 9 (Pointers and References) is assumed.

PART VIII

Linked Lists

From "a node with an arrow to the next one" to reversal, cycles, and merging

A linked list is the first data structure in this book where you build the structure yourself out of raw pieces, rather than reaching for a ready-made STL container. That makes it slightly more work up front, but it pays off: the pointer-manipulation habits you build here — following a chain, rewiring a link, running two pointers at different speeds — reappear constantly in trees (Part XI) and graphs (Part XIV).

Contents of Part VIII

- 1. What Is a Linked List?
- 2. Singly Linked Lists — Traversal, Insertion, Deletion
- 3. Reversing a Linked List
- 4. Fast and Slow Pointers — Middle Node and Cycle Detection
- 5. Doubly Linked Lists
- 6. Circular Linked Lists
- 7. Merging and Intersecting Lists
- 8. STL's `std::list`, and Mistakes and Complexity Recap
- Part VIII Revision, Cheat Sheet, and Interview Questions

CHAPTER 1

What Is a Linked List?

1.1 The problem with arrays that linked lists solve

In one sentence: a linked list is a chain of separately-allocated nodes, where each node stores a value and a pointer to the next node — so the data does not need to sit in one contiguous block of memory the way an array or vector's data does.

An array (or vector) stores its elements contiguously, which is exactly why indexing `a[i]` is $O(1)$ — the address is just `base + i * elementSize`. But that same contiguity is expensive to change: inserting a new element in the middle means shifting every element after it, an $O(n)$ operation. A linked list flips this trade-off: inserting or deleting a node once you're already there is $O(1)$, because you only need to rewire a couple of pointers — but you lose $O(1)$ indexing, since reaching the k -th node means following k pointers one at a time, starting from the front.

1.2 The node

Every linked list is built from a node struct: a value, plus one or more pointers to other nodes.

```
struct Node {  
    int data;  
    Node* next;  
    Node(int val) : data(val), next(nullptr) {}  
};
```

The smallest possible singly linked list node.

A list itself is usually just a pointer to its first node, conventionally called `head`. An empty list is represented by `head == nullptr`. The last node's next pointer is also `nullptr`, marking the end of the chain.

1.3 Arrays vs. linked lists — a direct comparison

Operation	Array / vector	Linked list
Access element at index i	$O(1)$	$O(n)$ - must walk from head
Insert/delete at the front	$O(n)$ - shift everything	$O(1)$
Insert/delete in the middle	$O(n)$ - shift elements	$O(1)$ once you're at the spot
Insert/delete at the back	$O(1)$ amortized (vector)	$O(1)$ if a tail pointer is kept
Memory layout	Contiguous	Scattered (each node malloc'd separately)
Cache performance	Good (contiguous)	Poor (pointer chasing)

Why vector usually wins in practice

In real competitive programming, vector is used far more often than a hand-rolled linked list, precisely because of that last row: cache performance. A linked list's $O(1)$ insertion advantage rarely outweighs vector's much better constant factor unless the problem specifically needs list-like pointer manipulation (as many interview and some DSA-focused CP problems do).

1.4 Building a tiny list by hand

```
Node* head = new Node(10);
head->next = new Node(20);
head->next->next = new Node(30);
// head -> 10 -> 20 -> 30 -> nullptr

for (Node* cur = head; cur != nullptr; cur = cur->next) {
    cout << cur->data << " ";
}
// prints: 10 20 30
```

Traversal always looks like this: start at head, follow next until nullptr.

⚠ **Memory ownership**

Every new `Node(...)` allocates memory that must eventually be freed with `delete`, or it leaks. In a contest this rarely matters (the program exits right after), but in real software it does — this is exactly the kind of manual memory management that modern C++ (smart pointers, containers) exists to avoid.

Practice — Chapter 1

Problem 1.1 — Build and print [Level 0]

Given n integers, build a singly linked list containing them in order, then print all values separated by spaces.

Hint 1: Keep a tail pointer while building so each new node can be appended in $O(1)$ instead of walking the whole list each time.

Approach: Create the first node as head and tail; for each remaining value, create a node, set `tail->next` to it, and advance tail.

Complexity: $O(n)$ time to build, $O(n)$ time to print, $O(n)$ space.

Problem 1.2 — Length of a list [Level 0]

Given the head of a singly linked list, count how many nodes it contains.

Approach: Walk from head to `nullptr` with a counter, incrementing once per node visited.

Complexity: $O(n)$ time, $O(1)$ extra space.

CHAPTER 2

Singly Linked Lists — Traversal, Insertion, Deletion

2.1 Insertion at the front

Inserting at the front is the simplest case: the new node's next points at the old head, and then head is updated to point at the new node.

```
void insertFront(Node*& head, int val) {  
    Node* newNode = new Node(val);  
    newNode->next = head;  
    head = newNode;  
}
```

Passing head by reference (Node&) lets this function update the caller's head pointer.*

⚠ Common mistake: forgetting the reference

If head is passed by value (just Node* head), reassigning it inside the function only changes the local copy — the caller's head pointer is left untouched. This is the exact pass-by-reference lesson from Part I, Chapter 8, applied to pointers.

2.2 Insertion at the end

```
void insertEnd(Node*& head, int val) {  
    Node* newNode = new Node(val);  
    if (head == nullptr) { head = newNode; return; }  
    Node* cur = head;  
    while (cur->next != nullptr) cur = cur->next;  
    cur->next = newNode;  
}
```

Without a tail pointer, appending requires walking the whole list first - $O(n)$.

✦ Practical tip

If you'll be appending many times, maintain a separate tail pointer alongside head so each append becomes $O(1)$ instead of $O(n)$. This is exactly how `std::list` (Chapter 8) and most production linked-list implementations work.

2.3 Insertion after a given node

```
void insertAfter(Node* prevNode, int val) {  
    if (prevNode == nullptr) return;  
    Node* newNode = new Node(val);  
    newNode->next = prevNode->next;  
    prevNode->next = newNode;  
}
```

Order matters: link the new node forward BEFORE overwriting `prevNode->next`.

⚠ Common mistake: wrong order of pointer updates

If you write `prevNode->next = newNode;` before setting `newNode->next`, you lose the reference to the rest of the list forever - the old `prevNode->next` is gone the moment you overwrite it.

2.4 Deletion

Deleting a node requires access to the node before it, since that is the only node whose next pointer needs to change to skip over the deleted node. This is why deleting the head is a special case: there is no "previous" node, so head itself must be updated instead.

```
void deleteValue(Node*& head, int val) {
    if (head == nullptr) return;
    if (head->data == val) {
        Node* toDelete = head;
        head = head->next;
        delete toDelete;
        return;
    }
    Node* cur = head;
    while (cur->next && cur->next->data != val)
        cur = cur->next;
    if (cur->next != nullptr) {
        Node* toDelete = cur->next;
        cur->next = cur->next->next;
        delete toDelete;
    }
}
```

Deleting the head is a special case; deleting any other node needs its predecessor.

Dry run: deleting 20 from 10 -> 20 -> 30

Step	cur	cur->next before	Action
1	head (10)	20	cur->next->data (20) matches val - stop searching
2	-	-	toDelete = node(20); cur->next = node(30); delete toDelete
Result	-	-	List is now 10 -> 30

2.5 Complexity of the basic operations

Operation	Complexity	Why
Insert at front	$O(1)$	Only head and one next pointer change
Insert at end (no tail ptr)	$O(n)$	Must walk to the last node first
Insert after a known node	$O(1)$	No search needed - node is given
Delete by value	$O(n)$	Must search for the value first
Access k-th element	$O(k)$	Must follow k pointers from head

Practice — Chapter 2

Problem 2.1 — Delete the middle node [Level 1]

Given the head of a singly linked list with an odd number of nodes, delete the middle node and return the new head.

Hint 1: You'll need the node just before the middle to rewire its next pointer.

Hint 2: Consider the fast/slow pointer idea previewed in this chapter's dry run.

Approach: Use two pointers, slow and fast, both starting at head; advance fast by two and slow by one each step. When fast reaches the end, slow (or the node just before it, tracked separately) is at the middle.

Complexity: $O(n)$ time, $O(1)$ extra space.

Problem 2.2 — Remove all occurrences of a value [Level 1]

Given the head of a singly linked list and a value `val`, remove every node whose data equals `val` (not just the first one found).

Key observation: A single pass with a trailing pointer handles this without needing to restart the search after each deletion.

Approach: Walk with a `cur` pointer and a `prev` pointer; whenever `cur->data == val`, unlink it via `prev->next = cur->next` and delete it, otherwise advance `prev` normally. Handle the head specially, since it has no `prev`.

Complexity: $O(n)$ time, $O(1)$ extra space.

CHAPTER 3

Reversing a Linked List

3.1 Why reversal is the classic pointer-manipulation exercise

Reversing a singly linked list means making every node's next pointer point backward instead of forward, and updating head to what used to be the last node. It is worth mastering thoroughly: the exact same three-pointer technique (previous, current, next) reappears throughout linked-list problems, and interviewers ask this constantly because getting the pointer order wrong is an easy, instructive mistake to make.

3.2 Iterative reversal

```
Node* reverseList(Node* head) {  
    Node* prev = nullptr;  
    Node* cur = head;  
    while (cur != nullptr) {  
        Node* nxt = cur->next; // save before overwriting cur->next  
        cur->next = prev; // reverse this node's pointer  
        prev = cur; // advance prev  
        cur = nxt; // advance cur, using the saved copy  
    }  
    return prev; // prev = old last node = the new head  
}
```

Three pointers, one saved value per step - the entire technique in nine lines.

Dry run: reversing 10 -> 20 -> 30

Step	prev	cur	nxt	Action taken
start	null	10	-	-
1	null	10	20	cur->next = prev (10->null); prev=10; cur=20
2	10	20	30	cur->next = prev (20->10); prev=20; cur=30
3	20	30	null	cur->next = prev (30->20); prev=30; cur=null
end	30	null	-	loop stops; return prev = 30 (new head)

⚠ Common mistake: overwriting next before saving it

Saving `nxt = cur->next` before writing `cur->next = prev` is the entire trick. Reversing those two lines loses the rest of the list forever - `cur->next` is your only remaining link to everything after `cur`.

3.3 Recursive reversal

Recursion offers a different, equally useful way to see the same problem: reverse everything after the head first, then fix up the head's own links.

```
Node* reverseRecursive(Node* head) {
    if (head == nullptr || head->next == nullptr)
        return head; // base case
    Node* newHead = reverseRecursive(head->next);
    head->next->next = head; // make the next node point back at head
    head->next = nullptr; // head is now last - must point to null
    return newHead; // deepest node is always the final new head
}
```

The base case is a list of 0 or 1 nodes - already "reversed" as-is.

Dry run: reversing 10 -> 20 -> 30 recursively

Calls go all the way down to the base case first, then unwind, fixing links on the way back up:

Call	head	Action on the way back up
reverseRecursive(10)	10	waits for the recursive call to return
reverseRecursive(20)	20	waits for the recursive call to return
reverseRecursive(30)	30	base case: head->next is null, return 30
back in call(20)	20	20->next->next=20 (30->20); 20->next=null
back in call(10)	10	10->next->next=10 (20->10); 10->next=null
final result	-	newHead = 30, giving 30 -> 20 -> 10 -> null

Iterative vs. recursive: space trade-off

Recursive reversal uses $O(n)$ call-stack space, versus $O(1)$ for the iterative version. In an interview, mention both; in a contest with a long list (n up to 10^5 or more), prefer the iterative version to avoid a stack overflow.

3.4 Reversing only part of a list

A common variant asks you to reverse only the nodes between positions left and right. The core idea: locate the node just before position left, then repeatedly move the node right after it to the front of the sub-range being reversed.

```
Node* reverseBetween(Node* head, int left, int right) {
    Node dummy(0); dummy.next = head;
    Node* prevLeft = &dummy;
    for (int i = 1; i < left; i++) prevLeft = prevLeft->next;
    Node* cur = prevLeft->next;
    for (int i = 0; i < right - left; i++) {
        Node* moved = cur->next;
        cur->next = moved->next;
        moved->next = prevLeft->next;
        prevLeft->next = moved;
    }
    return dummy.next;
}
```

A dummy node avoids special-casing `left == 1` (reversing starting at the head).

♦ Practical tip

A dummy (sentinel) node - a placeholder node before the real head - is a very common trick for avoiding head-is-special-case branches in linked-list code. It costs one extra $O(1)$ allocation and removes several edge cases.

Practice — Chapter 3

Problem 3.1 — Palindrome linked list [Level 2]

Given the head of a singly linked list, determine whether the sequence of values it represents is a palindrome.

Hint 1: Find the middle using fast/slow pointers.

Hint 2: Reverse the second half, then compare it against the first half.

Key observation: Combining the middle-finding technique from Chapter 2 with this chapter's reversal technique solves this in $O(1)$ extra space, without copying values into a separate array.

Approach: Find the middle, reverse the second half in place, walk both halves comparing values, then (optionally) reverse the second half back.

Complexity: $O(n)$ time, $O(1)$ extra space.

Problem 3.2 — Trace the recursion [Level 1]

For the list 5 -> 15 -> 25, trace reverseRecursive by hand, writing down the value of head at each call and the state of the list immediately after each unwind step.

Approach: This mirrors the dry run above with values 10, 20, 30 replaced by 5, 15, 25 - walk down to the base case at node 25, then fix links on the way back up.

CHAPTER 4

Fast and Slow Pointers — Middle Node and Cycle Detection

4.1 The fast/slow pointer idea

In one sentence: running two pointers through the same list at different speeds lets you answer questions about structure and position without knowing the list's length in advance, and without any extra memory. This single idea (also called "the tortoise and the hare") solves two classic problems: finding the middle node, and detecting a cycle.

4.2 Finding the middle node

Advance slow by one node and fast by two nodes on every step. When fast reaches the end (nullptr, or one step before it), slow is sitting exactly at the middle - because slow has covered exactly half the distance fast has covered.

```
Node* middleNode(Node* head) {  
    Node* slow = head;  
    Node* fast = head;  
    while (fast != nullptr && fast->next != nullptr) {  
        slow = slow->next;  
        fast = fast->next->next;  
    }  
    return slow; // middle node (later of 2, if length is even)  
}
```

fast moving twice as fast as slow is the entire mechanism.

Dry run: 10 -> 20 -> 30 -> 40 -> 50

Step	slow	fast	Loop condition still true?
start	10	10	yes
1	20	30	yes
2	30	50	yes
3	-	-	fast->next is null - loop stops; slow = 30 (the middle)

4.3 Cycle detection — Floyd's algorithm

A cycle exists when some node's next pointer eventually leads back to an earlier node instead of reaching nullptr. Floyd's cycle detection (the same tortoise-and-hare idea) answers "is there a cycle?" in $O(1)$ extra space: if fast ever catches up to and equals slow, there must be a cycle, since fast can only "lap" slow inside one.

```
bool hasCycle(Node* head) {
    Node* slow = head;
    Node* fast = head;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true; // fast lapped slow: a cycle
    }
    return false; // fast reached nullptr - no cycle
}
```

If there is no cycle, fast reaches the end normally, exactly like middleNode.

Why this actually works

If there is no cycle, fast simply reaches the end first, and the loop terminates normally - no false positives are possible. If there is a cycle, once both pointers have entered it, fast gains on slow by exactly one node per step (since fast moves 2 and slow moves 1, a relative gain of 1). Because the cycle has finite length, this gain must eventually bring fast to exactly the same node as slow - they cannot skip past each other on a single-directional chain of pointers.

4.4 Finding where the cycle begins

Once `hasCycle` detects a meeting point, a classic follow-up trick finds the exact starting node of the cycle: reset one pointer to head, keep the other at the meeting point, then advance both one step at a time - they meet again exactly at the cycle's start. This relies on a short piece of algebra relating the distance from head to the cycle's start and the distance already covered by the pointers, which is worth taking on faith here and revisiting once you're comfortable with the mechanics.

```
Node* cycleStart(Node* head) {
    Node* slow = head, *fast = head;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {
            Node* ptr = head;
            while (ptr != slow) { ptr = ptr->next; slow = slow->next; }
            return ptr; // the node where the cycle begins
        }
    }
    return nullptr; // no cycle
}
```

After the first meeting, resetting one pointer to head is the key extra step.

⚠ Common mistake: comparing values instead of pointers

Comparing node pointers with `==` checks whether they refer to the exact same node in memory, not whether their data values are equal. Cycle detection and middle-node logic both rely specifically on pointer identity, not on comparing `cur->data`.

Practice — Chapter 4

Problem 4.1 — Happy number (a non-list use of the same trick) [Level 3]

A number is 'happy' if repeatedly replacing it with the sum of the squares of its digits eventually reaches 1. Determine whether a given number is happy, without using a hash set to track seen values.

Hint 1: Treat 'apply the digit-square-sum function' as 'move to the next node' in an implicit, unwritten linked structure.

Hint 2: Run a slow pointer and a fast pointer (apply the function once vs. twice per step) exactly as in cycle detection.

Key observation: A number is unhappy precisely when this process enters a cycle instead of reaching 1 - so cycle detection directly answers the happy-number question.

Approach: Apply Floyd's algorithm with the 'next node' function being 'sum of squares of digits'; the number is happy exactly when the sequence reaches 1 before slow and fast ever meet at a non-1 value.

Complexity: $O(\log(\text{max value}))$ per step of the underlying sequence in practice.

Problem 4.2 — Middle node, even-length edge case [Level 1]

For a list of even length, e.g. 10 -> 20 -> 30 -> 40, which node does the middleNode function above return, and why?

Approach: It returns 30, the second of the two middle nodes - because fast exhausts the list (fast->next becomes null) exactly when slow has advanced two steps, landing it on the later of the two middles.

CHAPTER 5

Doubly Linked Lists

5.1 Adding a backward link

A doubly linked list's node stores two pointers - next and prev - so the list can be walked in either direction. This costs extra memory per node (one more pointer) but removes the need to track a separate "previous" pointer manually during deletion, and makes backward traversal possible in the first place.

```
struct DNode {
    int data;
    DNode* next;
    DNode* prev;
    DNode(int val) : data(val), next(nullptr), prev(nullptr) {}
};
```

The only structural difference from a singly linked node is the prev pointer.

5.2 Insertion at the front

```
void insertFront(DNode*& head, int val) {
    DNode* newNode = new DNode(val);
    newNode->next = head;
    if (head != nullptr) head->prev = newNode;
    head = newNode;
}
```

The old head's prev must now point back at the new node - easy to forget.

⚠ Common mistake: only updating next, forgetting prev

Forgetting `head->prev = newNode` (when head was not null) leaves the old head's prev pointing at garbage or the wrong node, silently corrupting backward traversal while forward traversal still looks completely correct.

5.3 Deletion of a given node

This is where a doubly linked list earns its extra memory: deleting a node you already have a pointer to is $O(1)$, with no search required, because the node itself already knows both its neighbors.

```
void deleteNode(DNode*& head, DNode* node) {
    if (node->prev != nullptr) node->prev->next = node->next;
    else head = node->next; // node was the head
    if (node->next != nullptr) node->next->prev = node->prev;
    delete node;
}
```

Compare this to singly linked deletion (Chapter 2), which needed a search for prev.

5.4 Complexity comparison

Operation	Singly linked	Doubly linked
Delete a given node (pointer in hand)	$O(n)$ - must find prev	$O(1)$
Traverse backward	Not possible directly	$O(n)$
Extra memory per node	1 pointer	2 pointers

Where this shows up: LRU cache

This $O(1)$ -delete-with-a-known-node property is exactly why an LRU cache (a very common interview question) is typically implemented with a doubly linked list plus a hash map: the map gives $O(1)$ lookup of the node, and the doubly linked list gives $O(1)$ removal and $O(1)$ re-insertion at the front.

Practice — Chapter 5

Problem 5.1 — Reverse a doubly linked list [Level 1]

Given the head of a doubly linked list, reverse it in place and return the new head.

Hint 1: At each node, next and prev simply need to be swapped.

Approach: Walk the list once; at each node swap its next and prev pointers, then move to what was originally next (now stored in prev); the new head is the old tail.

Complexity: $O(n)$ time, $O(1)$ extra space.

CHAPTER 6

Circular Linked Lists

6.1 What makes a list circular

In a circular linked list, the last node's next pointer points back to the first node instead of to nullptr, so there is no natural "end" to stop traversal at - the chain loops forever unless you explicitly track how far you've gone. Circular lists are used for round-robin scheduling, repeating buffers, and any situation modeled naturally as "wrapping around" back to the start.

6.2 Traversing a circular list safely

Since there is no nullptr to stop at, the standard pattern is a do-while loop that stops when you return to the starting node, not when you hit a null pointer.

```
void printCircular(Node* head) {  
    if (head == nullptr) return;  
    Node* cur = head;  
    do {  
        cout << cur->data << " ";  
        cur = cur->next;  
    } while (cur != head);  
}
```

do-while (checking the condition AFTER the body) is essential here - a while loop would need an extra flag to run even once.

⚠ Common mistake: while instead of do-while

Using a plain while (cur != head) loop instead of do-while would never execute the body at all, since cur starts out equal to head - printing nothing for a non-empty list. This is the one place in this book where do-while is the natural, correct choice rather than just an alternative syntax.

6.3 Insertion into a circular list

```
void insertCircular(Node*& head, int val) {
    Node* newNode = new Node(val);
    if (head == nullptr) {
        head = newNode;
        head->next = head; // points to itself: a 1-node circle
        return;
    }
    Node* cur = head;
    while (cur->next != head) cur = cur->next; // find the last node
    cur->next = newNode;
    newNode->next = head;
}
```

A single-node circular list must point to itself - a common edge case to miss.

6.4 Josephus-style problems

A very common circular-list application is the Josephus problem family: n people stand in a circle, and every k -th person is eliminated in turn until one remains. A circular linked list models this directly - simulate the elimination by walking around and removing nodes, which is far more natural here than trying to force an array to represent "wrapping around".

```
int lastRemaining(int n, int k) {
    Node* head = new Node(1);
    Node* cur = head;
    for (int i = 2; i <= n; i++) {
        cur->next = new Node(i);
        cur = cur->next;
    }
    cur->next = head; // close the circle
    cur = head;
    while (cur->next != cur) {
        for (int i = 1; i < k - 1; i++) cur = cur->next;
        Node* toRemove = cur->next;
        cur->next = toRemove->next;
        delete toRemove;
        cur = cur->next;
    }
    return cur->data;
}
```

The circle shrinks by one node per elimination until only `cur->next == cur` remains.

Practice — Chapter 6

Problem 6.1 — Detect whether a list is circular [Level 1]

Given a pointer to a node in a linked list (not necessarily the head), determine whether the list is circular.

Hint 1: This is exactly Floyd's cycle detection from Chapter 4, applied here as the definition of "circular" rather than as a bug to detect.

Approach: Run the standard fast/slow pointer cycle check; a circular list is, by construction, a list with a cycle that includes every node.

Complexity: $O(n)$ time, $O(1)$ extra space.

CHAPTER 7

Merging and Intersecting Lists

7.1 Merging two sorted lists

Given two already-sorted linked lists, merging them into one sorted list is a direct analog of the merge step in merge sort (Part VI), but done by rewiring pointers instead of copying into a new array - so it needs no extra node allocation at all, only reuse of the existing nodes.

```
Node* mergeTwoSorted(Node* a, Node* b) {
    Node dummy(0);
    Node* tail = &dummy;
    while (a != nullptr && b != nullptr) {
        if (a->data <= b->data) { tail->next = a; a = a->next; }
        else { tail->next = b; b = b->next; }
        tail = tail->next;
    }
    tail->next = (a != nullptr) ? a : b; // attach the leftovers
    return dummy.next;
}
```

A dummy node again removes the need to special-case which list contributes the very first node.

Dry run: merging 1 -> 3 -> 5 with 2 -> 4

Step	Compare	tail->next set to	Result so far
1	1 vs 2	1	1
2	3 vs 2	2	1 -> 2
3	3 vs 4	3	1 -> 2 -> 3
4	5 vs 4	4	1 -> 2 -> 3 -> 4
5	a exhausted	attach remaining b (5 from a's side is compared, then a exhausted, remaining a (5) attached)	1 -> 2 -> 3 -> 4 -> 5

7.2 Merging k sorted lists

A natural generalization: merge k sorted lists into one. Repeatedly merging two lists at a time, pairwise, is far better than merging them one-by-one into a growing result, because it keeps the total work at $O(N \log k)$ instead of $O(N k)$, where N is the total number of nodes across all lists.

```
Node* mergeKLists(vector<Node*> &lists) {
    if (lists.empty()) return nullptr;
    while (lists.size() > 1) {
        vector<Node*> merged;
        for (size_t i = 0; i + 1 < lists.size(); i += 2)
            merged.push_back(mergeTwoSorted(lists[i], lists[i+1]));
        if (lists.size() % 2 == 1) merged.push_back(lists.back());
        lists = merged;
    }
    return lists[0];
}
```

Pairwise merging, repeated, is the linked-list analog of merge sort's divide-and-conquer structure.

♦ Practical tip

An alternative approach uses a `priority_queue` (Part XIII) holding the current front node of each list, always extracting the minimum - also $O(N \log k)$, and often considered cleaner to implement once you're comfortable with heaps.

7.3 Finding the intersection point of two lists

Two singly linked lists are said to intersect if they share a common tail - at some node, they merge into the same chain. Because the lists may have different lengths before the intersection point, naively walking both from their heads at the same pace won't line them up correctly.

```
Node* getIntersection(Node* a, Node* b) {
    Node* pa = a;
    Node* pb = b;
    while (pa != pb) {
        pa = (pa == nullptr) ? b : pa->next;
        pb = (pb == nullptr) ? a : pb->next;
    }
    return pa; // the intersection node, or nullptr if none
}
```

Redirecting each pointer to the OTHER list's head once it hits null equalizes the distance each has to travel.

Why redirecting to the other list's head works

Let list A have length p before the intersection and list B have length q before it, with a shared tail of length c after. Pointer pa travels $p + c + q$ total steps by the time it reaches the intersection the second time around (its own list, then the other list's prefix); pointer pb travels $q + c + p$ - exactly the same total distance. Since both pointers cover an equal total distance before reaching the intersection node, they arrive there at the exact same step, whether or not the two original lists had different lengths.

Practice — Chapter 7

Problem 7.1 — Merge two sorted lists, trace it [Level 1]

Trace `mergeTwoSorted` by hand for lists $2 \rightarrow 6 \rightarrow 8$ and $1 \rightarrow 5 \rightarrow 9$, writing the state of the result list after each comparison.

Approach: Follow the same dry-run table format shown in this chapter, comparing elements pairwise and always advancing whichever pointer contributed the smaller value.

Problem 7.2 — Detect intersection without extra space [Level 2]

Two singly linked lists may or may not intersect. Determine whether they do, and if so, at which node, using $O(1)$ extra space (no hash set of visited nodes).

Key observation: A hash-set-based solution (Part VII) would work but costs $O(n)$ extra space; the two-pointer redirect technique in this chapter achieves the same result in $O(1)$ extra space.

Approach: Use the `getIntersection` two-pointer technique above; if the lists never intersect, both pointers become `nullptr` at the same time and the loop ends with `pa == pb == nullptr`.

Complexity: $O(p + q)$ time, $O(1)$ extra space, where p and q are the two list lengths.

CHAPTER 8

STL's `std::list`, and Mistakes and Complexity Recap

8.1 `std::list` — a ready-made doubly linked list

C++'s STL provides `std::list<T>`, a doubly linked list with the pointer-juggling already implemented for you. It supports $O(1)$ insertion and deletion at any position given an iterator, $O(1)$ `push_front/push_back`, but critically no random access - `list[i]` does not exist, unlike `vector`.

```
list<int> lst = {10, 20, 30};
lst.push_front(5); // 5 10 20 30
lst.push_back(40); // 5 10 20 30 40

auto it = lst.begin();
advance(it, 2); // move iterator forward 2 steps -  $O(k)$ , not  $O(1)$ !
lst.insert(it, 99); // 5 10 99 20 30 40
lst.erase(it); // removes 20 (erase takes an iterator, not value)
```

advance() on a list iterator is $O(k)$ - list trades random access for $O(1)$ mid-list insertion, it does not grant both.

8.2 When to actually reach for `std::list`

- You need to insert/erase frequently in the middle of a sequence, and you already hold iterators to the relevant positions (so you never need $O(n)$ traversal to get there).
- You need stable iterators - unlike `vector`, inserting into a list never invalidates existing iterators to other elements.
- You do NOT need random access or good cache locality - if you do, `vector` is almost always the better default in competitive programming, per Chapter 1's comparison.

A realistic expectation

In practice, `std::list` is used far less often in competitive programming than `vector`, precisely because most CP problems don't need frequent mid-sequence insertion with pre-held iterators - they need fast indexed access, which `vector` provides and `list` does not.

8.3 Common mistakes, gathered in one place

Mistake	Why it happens	Fix
Passing head by value into an insert/delete function	Reassigning the local copy doesn't affect the caller's pointer	Pass head by reference: <code>Node*&</code>
Overwriting <code>cur->next</code> before saving it	The rest of the list becomes unreachable the instant the pointer changes	Always save <code>nxt = cur->next</code> first, in reversal and similar code
Comparing node values instead of pointer identity	<code>cur->data == x</code> checks content; <code>cur == other</code> checks the actual node	Use <code>==</code> directly on pointers for identity (cycle/intersection code)
Using <code>while</code> instead of <code>do-while</code> on a circular list	<code>cur</code> starts equal to head, so a <code>while</code> condition is false immediately	Use <code>do-while</code> so the body runs at least once
Forgetting the single-node self-loop in a circular list	A list of one node must have <code>head->next == head</code> , not <code>nullptr</code>	Handle the first-insertion case explicitly
Memory leaks from deleted nodes	<code>delete</code> is easy to forget after unlinking a node	Always delete a node once it's fully unlinked

8.4 Complexity table — Part VIII

Operation	Singly linked	Doubly linked	std::list
Access by index	$O(n)$	$O(n)$	$O(n)$
Insert/delete at front	$O(1)$	$O(1)$	$O(1)$
Insert/delete given a node/iterator	$O(1)$	$O(1)$	$O(1)$
Search by value	$O(n)$	$O(n)$	$O(n)$
Reverse traversal	Not supported	$O(n)$	$O(n)$

Practice — Chapter 8 (mixed review)

Problem 8.1 — Remove duplicates from a sorted list [Level 1]

Given the head of a sorted singly linked list, remove all duplicate values so each value appears only once, keeping the list sorted.

Hint 1: Because the list is sorted, all duplicates of a value are already adjacent.

Approach: Walk with a single cur pointer; while `cur->next` has the same data as cur, skip and delete it; otherwise advance cur normally.

Complexity: $O(n)$ time, $O(1)$ extra space.

Problem 8.2 — Design choice [Level 2]

You are asked to implement an LRU cache with $O(1)$ get and put operations. Explain, using ideas from this Part and Part VII, what combination of data structures achieves this and why each piece is necessary.

Key observation: A hash map alone gives $O(1)$ lookup but no ordering by recency; a linked list alone gives ordering by recency but $O(n)$ lookup by key.

Approach: Combine an `unordered_map` for $O(1)$ lookup with a doubly linked list ordered by recency of use, so a node can be unlinked and moved to the front in $O(1)$ once found via the map.

Complexity: $O(1)$ average time per get/put operation.

PART VIII — MASTER REVISION

Linked Lists

Must Remember

- A linked list trades an array's $O(1)$ indexing for $O(1)$ insertion/deletion once you're already at the right node - it does not make everything faster, only certain operations.
- Always pass head by reference (`Node*&`) into any function that might reassign it.
- Reversal's core trick: save `nxt = cur->next` before overwriting `cur->next`, every time.
- Fast/slow pointers (tortoise and hare) solve middle-node finding and cycle detection with $O(1)$ extra space - the same mechanism, used two different ways.
- Doubly linked lists give $O(1)$ deletion given a node pointer (no search for prev needed) at the cost of one extra pointer per node - this is the basis of an LRU cache.
- Circular lists need a do-while loop (not while) for traversal, since `cur` starts equal to head.
- `std::list` gives you a ready-made doubly linked list, but still has no random access - vector remains the default choice unless you specifically need list's properties.

Common Mistakes Recap

Mistake	Fix
Passing head by value	Pass by reference: Node*&
Overwriting next before saving it	Save <code>nxt = cur->next</code> first
Comparing values instead of pointer identity	Use <code>==</code> on pointers directly
while instead of do-while on circular lists	Use do-while
Missing self-loop for a single-node circular list	<code>head->next = head</code> explicitly
Leaking deleted nodes	Always delete after unlinking

Complexity Table

Operation	Array/vector	Linked list
Access by index	$O(1)$	$O(n)$
Insert/delete at front	$O(n)$	$O(1)$
Insert/delete given a position/node	$O(n)$	$O(1)$
Middle node / cycle detection	trivial with indexing	$O(n)$, $O(1)$ space via fast/slow

Interview-Style Questions

- 1 Why is inserting at the front of a linked list $O(1)$, while inserting at the front of a vector is $O(n)$?
- 2 Walk through why the fast/slow pointer technique correctly detects a cycle, and why it never produces a false positive when no cycle exists.
- 3 What extra capability does a doubly linked list provide over a singly linked list, and what is the memory cost of that capability?
- 4 Why does merging two sorted linked lists need no extra node allocation, unlike merging two sorted arrays into a new array?

- 5 Why does `std::list` not support `operator[]`, even though it supports fast insertion anywhere given an iterator?

Cheat Sheet — Part VIII

```
// Node definitions
struct Node { int data; Node* next; };
struct DNode { int data; DNode *next, *prev; };

// Traversal
for (Node* cur = head; cur; cur = cur->next) { /* ... */ }

// Reversal (iterative)
Node* prev = nullptr, *cur = head;
while (cur) {
    Node* nxt = cur->next;
    cur->next = prev;
    prev = cur; cur = nxt;
}
head = prev;

// Fast / slow pointers (middle node)
Node *slow = head, *fast = head;
while (fast && fast->next) {
    slow = slow->next; fast = fast->next->next;
}
```

```
// Fast / slow pointers (cycle detection)
slow = head; fast = head;
while (fast && fast->next) {
    slow = slow->next; fast = fast->next->next;
    if (slow == fast) { /* cycle found */ break; }
}

// Merge two sorted lists
Node dummy(0), *tail = &dummy;
while (a && b) {
    if (a->data <= b->data) { tail->next = a; a = a->next; }
    else { tail->next = b; b = b->next; }
    tail = tail->next;
}
tail->next = a ? a : b;

// Two-pointer intersection
Node *pa = a, *pb = b;
while (pa != pb) {
    pa = pa ? pa->next : b;
    pb = pb ? pb->next : a;
}
```

Coming up in Part IX

The next file is Part IX — Stacks, Queues and Deques: implementations (array-based, linked-list-based, and STL), the monotonic stack pattern, circular queues, and where each of these structures shows up constantly in competitive programming.